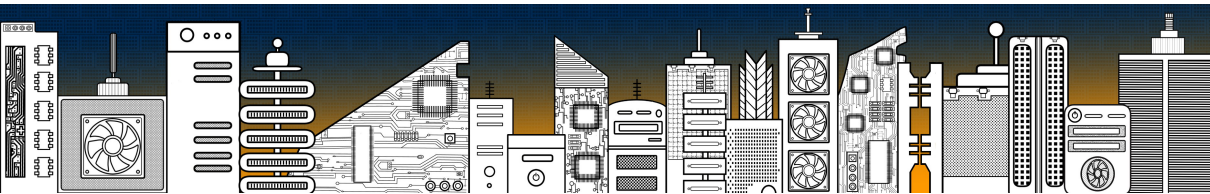
universität innsbruck



# Rechnerarchitektur

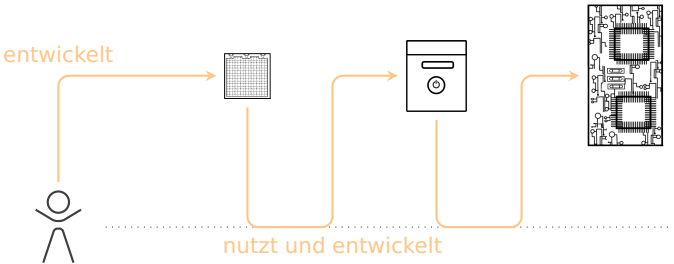
Einführung

Univ.-Prof. Dr. Rainer Böhme

Wintersemester 2026/27 · 7. Oktober 2026

# Technischer Fortschritt

Existierende Rechner sind Werkzeuge zum Entwurf neuer Rechner.



# Ziele dieser Lehrveranstaltung

- Vermittlung von Grundwissen über den **Entwurf** von **Digitalschaltungen**
- Verständnis des **Aufbaus** und der **Arbeitsweise** von Hardware
- Verständnis von **Architekturprinzipien** und **Organisationsformen** moderner Rechner
- Einsicht in das Zusammenspiel von Hardware und Software
- Erstellung von **maschinennahen Programmen** am Beispiel der ARM-Architektur
- Lernen von Ansätzen zur **Bewertung** und zum Vergleich von Rechnersystemen



# Organisation dieses Moduls

**Vorlesung**

- Zeit: mittwochs, 12:00–13:30 Uhr
- Ort: Hörsaal A (Technik)
- Umfang: 3 ECTS-AP European Credit Transfer System-Anrechnungspunkte

**Übung**

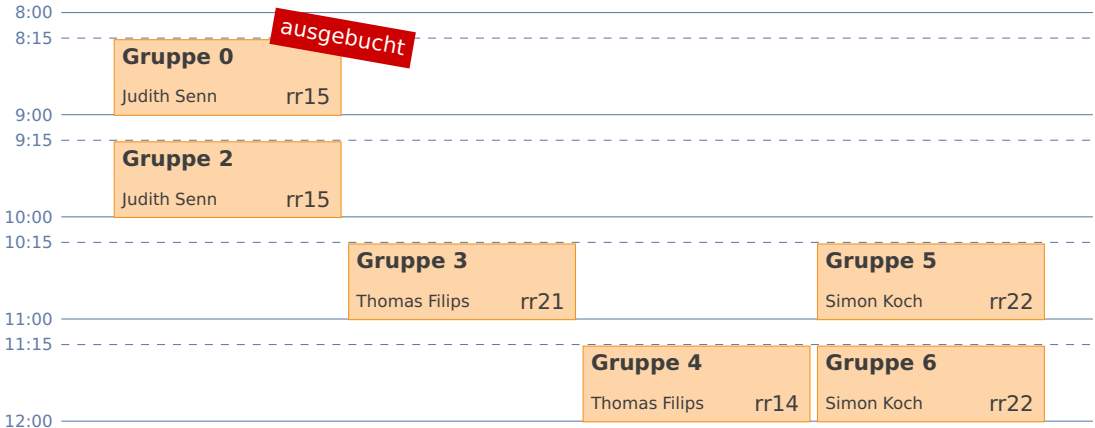
- Zeit: donnerstags, 08:15, 09:15, 10:15, 11:15 Uhr, ab 8.10.
- Ort: Rechnerräume im Untergeschoss des Architekturgebäudes
- Umfang: 2 ECTS-AP – Gruppe prüfen, Anwesenheitspflicht !

**Tutorium**

- Zeit: mittwochs, 09:00–09:45 Uhr, ab 14.10., freiwillig
- Ort: Rechnerraum rr20
- Tutorin: Tanja Plieth Tanja.Plieth@student.uibk.ac.at

# Plan der Übungsgruppen

## Donnerstag




Wer noch in eine andere Gruppe wechseln möchte, schreibt heute bis 17 Uhr eine E-Mail mit seinem/ihrem Wunschtermin an: `rechnerarchitektur-informatik@uibk.ac.at`. Wir bestätigen den Wechsel nach Möglichkeit noch heute Abend.

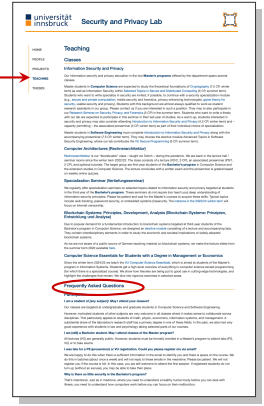
# Bewertung

## Vorlesung

- Schriftliche Klausur (Theorie)
- 1. Klausur 10.02.2027, 12:00–14:00 Uhr, HS A+B (Technik)
- Anmeldung bis zum 27.01.2027 möglich.
- Zwei weitere Termine: 31.03.2027 und im September

## Übung

- Wöchentlich computergestützte Tests
- Übungsaufgaben zur Vorbereitung
- Anerkennung für aktive Mitarbeit
- Weitere Details morgen in der Übung


universität innsbruck Security and Privacy Lab

HOME
PEOPLE
PROJECTS
TEACHING
THESES

Teaching

Classes

Information Security and Privacy

Our information security and privacy education in the two Master's programs offered by the department spans several classes.

Master students in Computer Science are expected to study the theoretical foundations of Cryptography (7.5 ECTS) either in the Bachelor's program or in the Master's program. Cryptography is a prerequisite for many other classes. Students who want to write specialize in security can choose, if possible, to continue with a security specialization module (e.g., secure and private computation, media security and forensics, privacy-enhancing technologies, game theory for security, or machine learning and security). Students can also choose to write their thesis in our group. Please contact us if you are interested in such a position. They may to play participate in our research seminar, where we talk (unofficially) about recent research papers. Students who have done well in our classes and with our lab are expected to participate in the seminar in the last year of studies, via a seminar paper, students interested in writing their master's thesis with us are also expected to participate in the seminar. Finally, we offer a Master's Project (15 ECTS) for hands-on / capacity-permitting – the associated proseminar (3 ECTS) which tend to be part of their individual choice of specialization.

Master students in Software Engineering must complete Introduction to Information Security and Privacy along with the accompanying proseminar (7 ECTS) in their terms. They may choose the elective module Advanced Topics in Software Security Engineering, where we take contributes the To Secure Programming (6 ECTS seminar term).

Computer Architecture (Pro/Forensic/Artistic)

Rechnerarchitektur is our "flexibilization" class - taught via Sokrin - during the pandemic. We are back in the lecture hall – Rechnerarchitektur is a lecture with 6 ECTS, and optional tutorials. The target group are first-year students of the Bachelor's program in Computer Science and Business Informatics. The lecture and exercise modules with a written exam and the proseminar in grasshoppers on weekly online quizzes.

Specialization Seminar (Vertiefungsseminar)

We regularly offer specialization seminars on selected topics related to information security and privacy beyond all students in the third year of the Bachelor's program. These seminars do not require the hard-core deep understanding of mathematical foundations that is required for the lecture and exercise modules. Topics in the past have included web tracking, password security, or embedded systems (especially. The Interface of the EMBSYS winter term will focus on internet censorship.

Blockchain Systems: Principles, Development, Analysis (Blockchain-Systeme: Prinzipien, Entwicklung, Analyse) (Seminar)

Blockchain systems are a fascinating technology that, in its technical systems targeted at third-year students of the Bachelor's program in Computer Science, we designed as elective module consisting of a lecture and accompanying tasks. The lecture introduces the necessary elements in order to study the economic and societal implications of widely adopted blockchain systems.

As we cannot assess of a public source of German-teaching material on blockchain systems, we make the lecture slides from the summer term 2020 available here.

Computer Science Essentials for Students with a Degree in Management or Economics

Since the winter term 2020/20 we teach the VU Computer Science Essentials, which is aimed at students of the Master's programs in Management, Economics and Social Sciences. The lecture is intended to convey basic computer science principles and also contributes to specialized courses. We know how Theories are being put to good use in cutting-edge technologies, and highlight the challenges that arise. We also may organize workshops in selected areas.

Frequently Asked Questions

I am a student of [any subject]. May I attend your classes?

Our classes are targeted at undergraduate and graduate students in Computer Science and Software Engineering.

However, motivated students of other subjects are very welcome in all classes where it makes sense to collaborate across disciplines. This primarily applies to students of other computer science subjects, Mathematics, and Management. A background in Mathematics is helpful for many of our classes, especially in Cryptography and Theory. In the past, we also had very good experiences with students in law and psychology taking selected parts of our courses.

I am [still] a Bachelor student. May I attend classes of the Master program?

All lectures (VO) are generally public. However, students must be formally enrolled in a Master's program to attend class (PS), seminar (SE), or project (PR).

I was late to / (I'll) (procrastinate) on PK registration. Could you please register me via email?

We are happy to do this when there is sufficient information in the email to identify you and there is space on the course. We do not do this automatically for all students. If you are late to registration, please be prepared to attend the first session. If you register late (the course is full, in this case, you are still welcome to attend the first session. Empowered students do not turn up (without an excuse), you may be able to take their place.

Why is there so little security in the Bachelor's programs?

That's intentional. Just as in medicine, where you need to understand a healthy human body before you can deal with illness, you need to understand how computers work before you can "rescue" them malfunction.

Lesen Sie auch die FAQ zu Prüfungen auf unserer Webseite unter „Teaching“.

# Vorlesungsmaterialien

**Folien**

- Die Folien stehen nach der Vorlesung im OpenOlat LMS („OLAT“)
- Verwenden Sie die Seitenzahlen als Referenz für Ihre Mitschrift.

**MOOC**

- Auf iMooX finden Sie 66 Videoeinheiten zu zentralen Aspekte der Vorlesung.
- Besuchen Sie `https://imoox.at/course/RA` und verwenden Sie Ihr eduID-Login.

**Lehrbücher**

- D. A. Patterson* and J. L. Hennessy*, Computer Organization and Design, 5. Auflage, Morgan Kaufmann, 2014. (4. Auflage auch auf Deutsch erschienen)
- D. W. Hoffmann, Grundlagen der Technischen Informatik, 3. Auflage, Hanser, 2013.
- W. Oberschelp und G. Vossen, Rechneraufbau und Rechnerstrukturen, 10. Auflage, Oldenbourg, 2006.

* Turing-Award 2017 (der „Nobelpreis der Informatik“): `https://www.acm.org/hennessy-patterson-turing-lecture`

# Ihr Rechnerarchitektur-Team

## Wintersemester 2026/27

Professor



Univ.-Prof. Dr.
Rainer Böhme

Proseminarleiter:innen



Thomas
Filips



Simon
Koch



Judith
Senn

Tutorin



Tanja
Plieth

## Danksagung

Die Folieninhalte basieren teilweise auf den Materialien meiner Vorgänger:
Univ.-Prof. Drs. Matthias Harders, Falko Dressler, Alfred Strey

Bei der Gestaltung aller Materialien haben in vergangenen Jahren mitgewirkt:
Gloria Dzida, Dr. Felix Erlacher, Dr. Michael Fröwis, Dr. Nora Hofer, Dr. Patrik Keller, Verena Lachner, Dr. Benedikt Lorch, Kristina Magnussen, Alexander Schlögl, Dr. Pascal Schöttle

# Bemerkungen zur akademischen Ausbildung

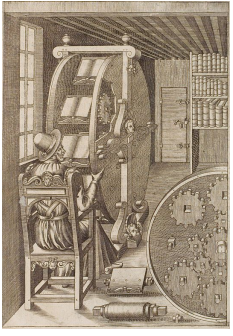
**Rolle der Universität**

- Kombination von Forschung und Lehre
- Innovation = Idee + Ausführung
- **Lernziel:** Spaß an systematischem Denken
- „Rohstoff für Wachstum“

**Bedeutung des „Stoffs“ in der Lehre**

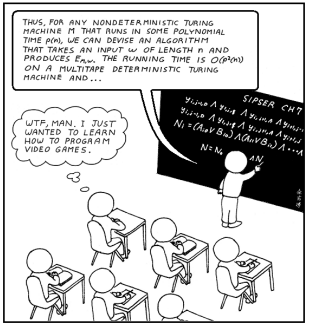
- Vermittlung von Prinzipien und Denkmethoden, die Innovation fördern
- Bekannte, in der Praxis verbreitete Lösungen sind Anschauungsmaterial
- Praxisrelevantes Faktenwissen als Nebeneffekt (veraltet schnell)



Bücherrad, Agostion Ramelli 1588, Bild: Wikipedia

# Konsequenzen für die Lehre im Bachelorstudium



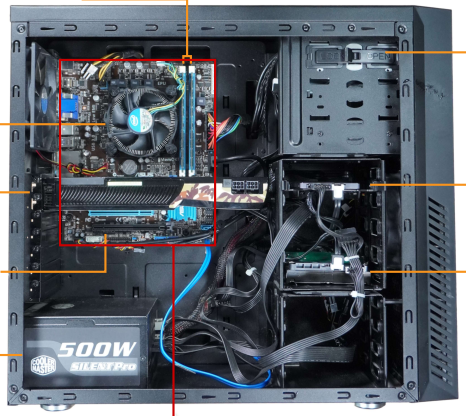

**Voraussetzungen**

- **Grundlagen** des Fachs
- **Austausch:** Diskurs, Gastvorträge, Auslandssemester, etc.
- Fachspezifische **Kommunikation**: Terminologie und Konventionen, Fremdsprachen, wissenschaftliche Literatur („Papers“)
- Fachspezifische **Fertigkeiten**, z. B. Programmieren
- **Ethik:** Standards guter wissenschaftlicher Praxis (z. B. Reproduzierbarkeit)

→ Fehler passieren trotzdem.

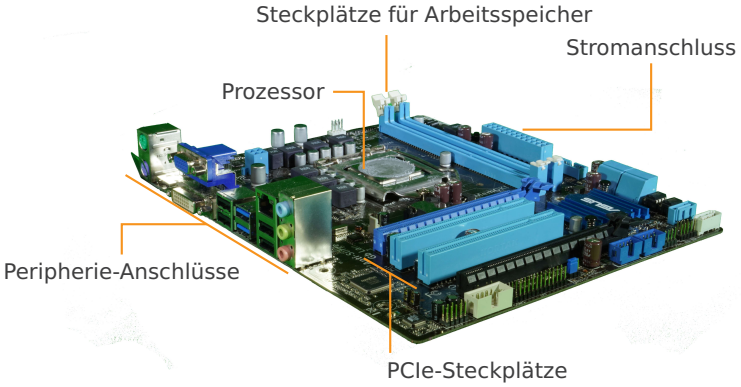
Quelle für Illustration: xkcd.com

# Innenansicht eines Desktop-PCs

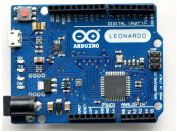


# Hauptplatine

(engl. Motherboard)



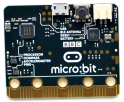
# Populäre Einplatinenrechner




**Arduino**
8-Bit-Atmel AVR CPU, 16 MHz
32 KB Flash-Speicher, USB
ca. 20 Euro (Nachbauten ab 10 Euro)



**Raspberry Pi**
32-Bit-ARM Cortex-A CPU, 1.2 GHz
1 GB Arbeitsspeicher, HDMI, Ethernet
ca. 40 Euro

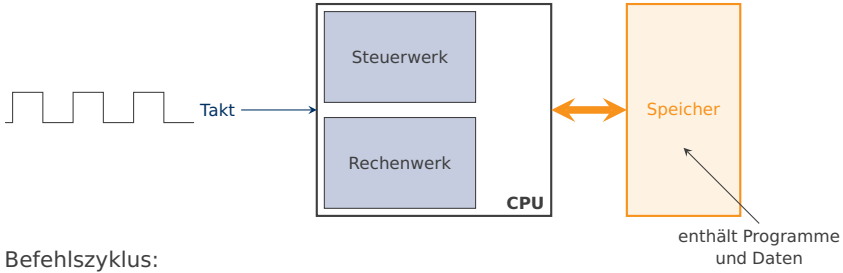



2 €

**Micro Bit**
32-Bit-ARM Cortex-M0 CPU, 16 MHz
256 KB Flash-Speicher, USB, Bluetooth
Sensoren, LED-Matrix, Tasten
ca. 20 Euro

# Prozessor

(engl. Central Processing Unit, CPU)




In jedem Befehlszyklus:

- Holen einer Instruktion aus dem Speicher
- Dekodieren im Steuerwerk
- Austausch benötigter Daten (= Operanden) mit dem Speicher
- Ausführen im Rechenwerk

**Die CPU ist die Hauptkomponente jedes Rechners.**

# Beispiel

Programm in der Programmiersprache C

```c
#include <stdio.h>

  int main(void)
  {
    printf("Was ist Software?\n");

    return 0;
  }
```

# Beispiel

## Maschinennahes Assembler-Programm (x86-Befehlssatz, erzeugt vom Compiler)

```
__text:00001F44                 call    _exit
__text:00001F49                 hlt
__text:00001F49 start           endp
__text:00001F49
__text:00001F4A
__text:00001F4A ; =============== S U B R O U T I N E =======================================
__text:00001F4A
__text:00001F4A ; Attributes: bp-based frame
__text:00001F4A
__text:00001F4A                 public _main
__text:00001F4A _main           proc near               ; CODE XREF: start+30↑p
__text:00001F4A                 push    ebp
__text:00001F4B                 mov     ebp, esp
__text:00001F4D                 push    ebx
__text:00001F4E                 sub     esp, 14h
__text:00001F51                 call    $+5
__text:00001F56                 pop     ebx
__text:00001F57                 lea     eax, (aWasIstSoftware - 1F56h)[ebx] ; "Was ist Software?"
__text:00001F5D                 mov     [esp], eax      ; char *
__text:00001F60                 call    _puts
__text:00001F65                 mov     eax, 0
__text:00001F6A                 add     esp, 14h
__text:00001F6D                 pop     ebx
__text:00001F6E                 leave
__text:00001F6F                 retn
__text:00001F6F _main           endp
__text:00001F6F
__text:00001F6F __text          ends
__text:00001F6F
__cstring:00001F70 ; ===========================================================================
__cstring:00001F70
__cstring:00001F70 ; Segment type: Pure data
__cstring:00001F70 __cstring    segment byte public 'DATA' use32
__cstring:00001F70              assume cs:__cstring
__cstring:00001F70              ;org 1F70h
__cstring:00001F70 aWasIstSoftware db 'Was ist Software?',0 ; DATA XREF: _main+D↑o
__cstring:00001F70 __cstring    ends
__cstring:00001F70
__symbol_stub:00001F82 ; ===========================================================================
__symbol_stub:00001F82
__symbol_stub:00001F82 ; Segment type: Pure code
__symbol_stub:00001F82 __symbol_stub segment word public 'CODE' use32
__symbol_stub:00001F82              assume cs:__symbol_stub
__symbol_stub:00001F82              ;org 1F82h
```

# Beispiel

## Darstellung als Speicherauszug (Hexadezimaldarstellung der Byte-Werte)

```
00001F0A  00 00 6A 00 89 E5 83 E4  F0 83 EC 10 8B 5D 04 89  ..j..........]..
00001F1A  1C 24 8D 4D 08 89 4C 24  04 83 C3 01 C1 E3 02 01  .$.M..L$........
00001F2A  CB 89 5C 24 08 8B 03 83  C3 04 85 C0 75 F7 89 5C  ..\$........u..\
00001F3A  24 0C E8 09 00 00 00 89  04 24 E8 39 00 00 00 F4  $........$.9....
00001F4A  55 89 E5 53 83 EC 14 E8  00 00 00 00 5B 8D 83 1A  U..S........[...
00001F5A  00 00 00 89 04 24 E8 23  00 00 00 B8 00 00 00 00  .....$.#........
00001F6A  83 C4 14 5B C9 C3 57 61  73 20 69 73 74 20 53 6F  ...[..Was ist So
00001F7A  66 74 77 61 72 65 3F 00  FF 25 1C 20 00 00 FF 25  ftware?..%. ...%
00001F8A  20 20 00 00 68 18 20 00  00 FF 25 14 20 00 00 90    ..h. ...%. ....
00001F9A  68 0C 00 00 00 E9 EA FF  FF FF 68 00 00 00 00 E9  h.........h.....
00001FAA  E0 FF FF FF 01 00 00 00  00 00 1C 00 00 00 00 00  ................
00001FBA  00 00 1C 00 00 00 00 00  00 00 1C 00 00 00 02 00  ................
00001FCA  00 00 00 00 00 00 34 00  00 00 34 00 00 00 F9 0F  ......4...4.....
00001FDA  00 00 00 00 00 00 34 00  00 00 03 00 00 00 0C 00  ......4.........
00001FEA  01 00 10 00 01 00 00 00  00 00 00 00 00 00        ..............
00002000  00 10 00 00 24 20 00 00  28 20 00 00 2C 20 00 00  ....$ ..( .., ..
00002010  30 20 00 00 E8 31 00 00  00 00 00 00 E0 31 00 00  0 ...1.......1..
00002020  E4 31 00 00 00 00 00 00  00 00 00 00 00 00 00 00  .1..............
00002030  00 00 00 00                                      ....
00003000  11 40 64 79 6C 64 5F 73  74 75 62 5F 62 69 6E 64  .@dyld_stub_bind
00003010  65 72 00 51 72 14 90 00  72 1C 11 40 5F 65 78 69  er.Qr...r..@_exi
00003020  74 00 90 00 72 20 11 40  5F 70 75 74 73 00 90 00  t...r .@_puts...
00003030  00 02 5F 00 0C 73 74 61  72 74 00 4B 00 04 5F 00  .._.start.K.._.
00003040  27 6D 61 69 6E 00 50 4E  58 41 72 67 00 55 65 6E  'main.PNXArg.Uen
00003050  76 69 72 6F 6E 00 67 00  02 6D 68 5F 65 78 65 63  viron.g..mh_exec
00003060  75 74 65 5F 68 65 61 64  65 72 00 47 5F 70 72 6F  ute_header.G_pro
00003070  67 6E 61 6D 65 00 6C 02  00 00 00 03 00 8C 1E 00  gname.l.........
00003080  03 00 CA 1E 00 00 02 63  00 5D 76 00 62 03 00 A4  .......c.]v.b...
00003090  20 00 03 00 A8 20 00 03  00 AC 20 00 03 00 B0 20  .... .... .... 
000030A0  00 00 00 00 02 00 00 00  1E 04 00 00 8E 1F 00 00  ................
000030B0  10 00 00 00 0E 06 00 00  00 20 00 00 17 00 00 00  ......... ......
000030C0  0F 09 00 00 24 20 00 00  1F 00 00 00 0F 09 00 00  ....$ ..........
000030D0  28 20 00 00 27 00 00 00  0F 09 00 00 30 20 00 00  ( ..'.......0 ..
000030E0  33 00 00 00 03 00 10 00  00 10 00 00 47 00 00 00  3...........G...
000030F0  0F 09 00 00 2C 20 00 00  50 00 00 00 0F 01 00 00  ...., ..P.......
00003100  4A 1F 00 00 56 00 00 00  0F 01 00 00 0C 1F 00 00  J...V...........
00003110  5C 00 00 00 01 00 01 01  00 00 00 00 62 00 00 00  \...........b...
00003120  01 00 01 01 01 00 00 00  68 00 00 00 01 00 00 01  ........h.......
00003130  00 00 00 00 09 00 00 00  0A 00 00 00 00 00 00 40  ...............@
00003140  00 00 00 40 09 00 00 00  0A 00 00 00 20 00 20 73  ...@........ . s
00003150  74 75 62 20 68 65 6C 70  65 72 73 00 5F 70 76 61  tub helpers._pva
00003160  72 73 00 5F 4E 58 41 72  67 63 00 5F 4E 58 41 72  rs._NXArgc._NXAr
00003170  67 76 00 5F 5F 5F 70 72  6F 67 6E 61 6D 65 00 5F  gv.___progname._
00003180  5F 6D 68 5F 65 78 65 63  75 74 65 5F 68 65 61 64  _mh_execute_head
```

# Zahlendarstellung




**Basis:** Anzahl der verwendeten Ziffern in einem Stellenwertsystem

In Digitalschaltungen bietet sich die Basis 2 an: $\{0, 1\}$

$$(101101)_2 = (45)_{10} \quad \leftarrow \text{Basis 10 i. d. R. weggelassen}$$

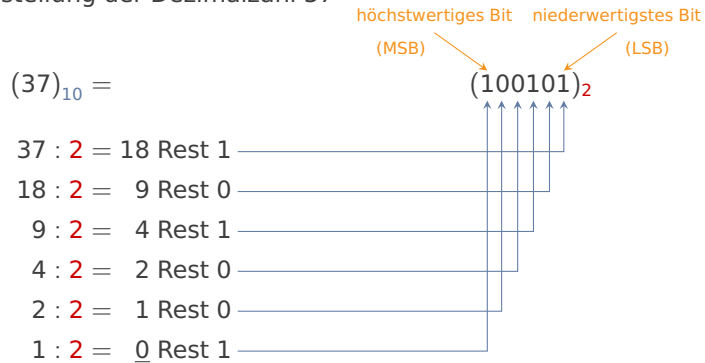
$$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 \cdot 10^1 + 5 \cdot 10^0$$

$$1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 4 \cdot 10 + 5 \cdot 1$$

Die Zahlendarstellung zur Basis 2 nennt man binär.

# Umrechnung von dezimal nach binär

**Gesucht:** Binärdarstellung der Dezimalzahl 37




Die niederwertigste Ziffer (Bit) gibt an, ob die Zahl gerade (0) oder ungerade (1) ist.

# Hexadezimalzahlen

Stellenwertsystem zur Basis 16: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}, \mathtt{e}, \mathtt{f}\}$

**Grund:** 8 Bit = 1 Byte, $2^4 = 16 \rightarrow$ Darstellung eines Bytes in zwei Ziffern

$$(\underbrace{1101}_{\mathtt{d}}\ \underbrace{0110}_{6})_2$$

Umrechnung hexadezimal $\rightarrow$ dezimal: $(\mathtt{d6})_{16} = 13 \cdot 16^1 + 6 \cdot 16^0 = (214)_{10}$

Umrechnung dezimal $\rightarrow$ hexadezimal: Wiederholt durch 16 teilen und Reste notieren.

**Konventionen**

Verwechslungsgefahr, falls Hexadezimalzahlen keine Ziffern a–f enthalten.

Kennzeichnung durch Präfix 0x oder Suffix h. Bsp.: 0x0400 = 400h = 1024

# Hörsaalfragen

Wir verwenden das Audience Response System der Uni Innsbruck.

- Benutzen Sie Ihr Smartphone.
- Scannen Sie den QR-Kode.
- Tippen Sie auf „Hörsaalfragen“.



**Fragen**

1. Welche Ziffernfolge entspricht der Dezimalzahl 13 in Binärdarstellung?
2. Welche Zeichenfolge entspricht der Dezimalzahl 13 im Hexadezimalsystem?

Alternativer Zugang ohne QR-Kode: `https://ars.uibk.ac.at` mit „Raumnummer“ **24 82 94 16**.

# Beispiel einer historischen CPU

MOS 6502 (1975)

- Apple II
- Atari 800 XL
- Commodore C64 (als 6510)
- Nintendo NES (leicht reduziert)



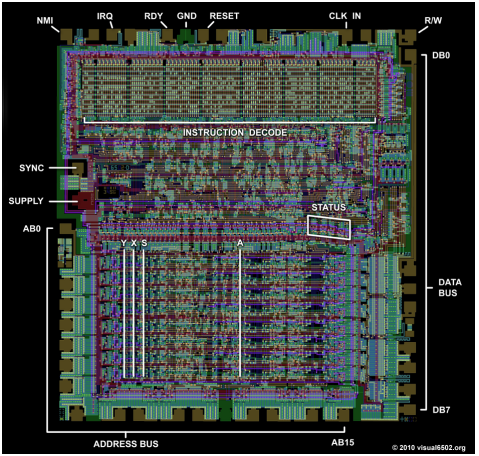
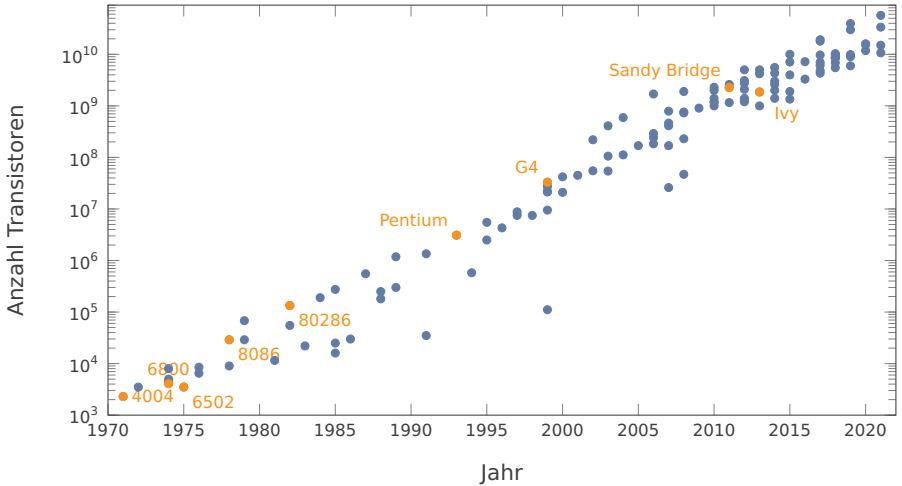




Bild: visual6502.org

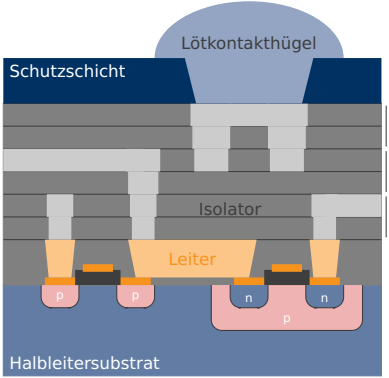
# Anzahl der Transistoren in Mikroprozessoren




Datenquelle: Wikipedia 2022

# Aufbau von Mikrochips

Schnitt durch einen “Die” (engl.), Herstellung im Planarprozess




**Packaging**
Testen, Vereinzeln, Drahtbonden

**Back-end of line** (BEOL)
Beschichten, Belichten, Ätzen, . . .

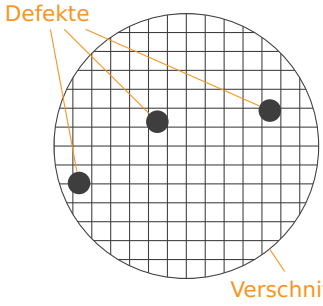
Ca. 10 Verdrahtungsebenen

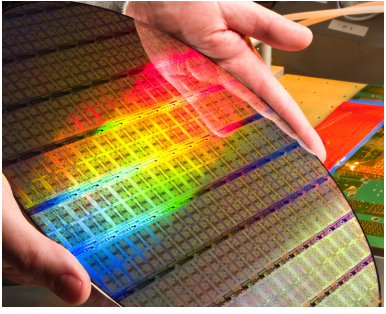
**Front-end of line** (FEOL)
Beschichten, Belichten,
Ätzen, Dotieren, . . .

Moderne Produktionsprozesse haben bis zu 500 Schritte.

# Wafer

(engl. für „Waffel“ oder „Oblate“)



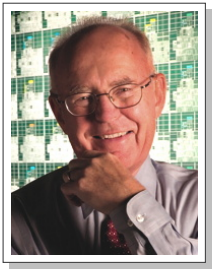
- Wafer-Durchmesser steigt: 50 mm (1970) – 450 mm (2025)
- Strukturgröße sinkt: 10 $\mu$m (1971) – 2 nm (2026)
- Die-Fläche steigt: 12 mm$^2$ (1971) – 826 mm$^2$ (2020)

Bildquelle: IBM (300 mm-Wafer)

# Gordon Moore

*“The complexity for minimum component costs has increased at a rate of roughly a factor of two per year. Certainly over the short term this rate can be expected to continue, if not to increase.”*

Electronics Magazine, 1965

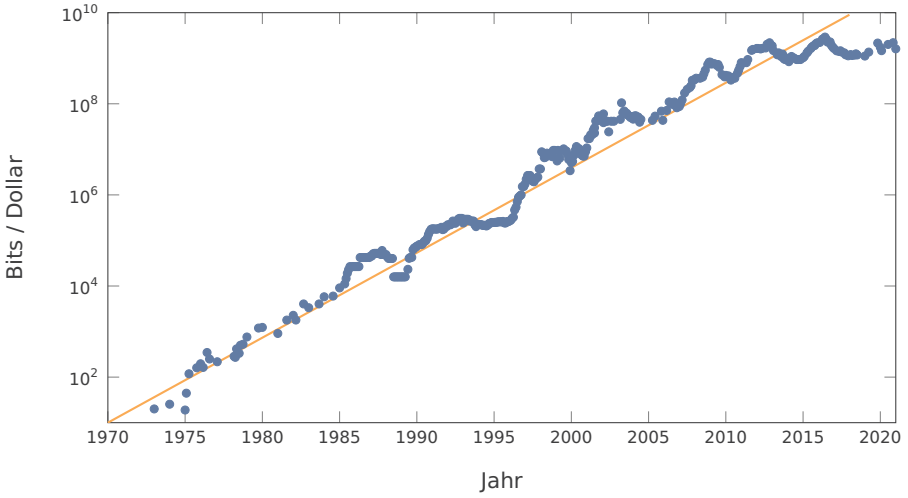


2005

Gordon E. Moore, 1929–2023, Mitgründer von Intel
Urheber des Moore’schen Gesetzes

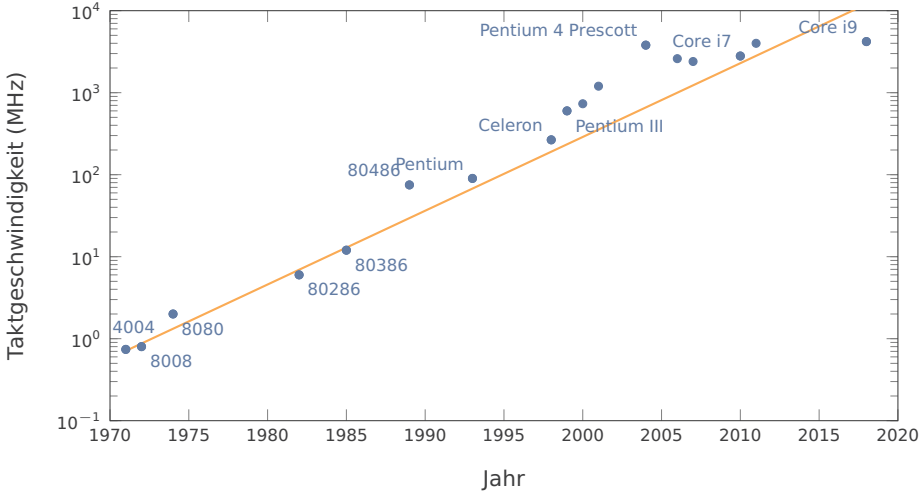
Bild: Intel

# Entwicklung der Größe des Arbeitsspeichers



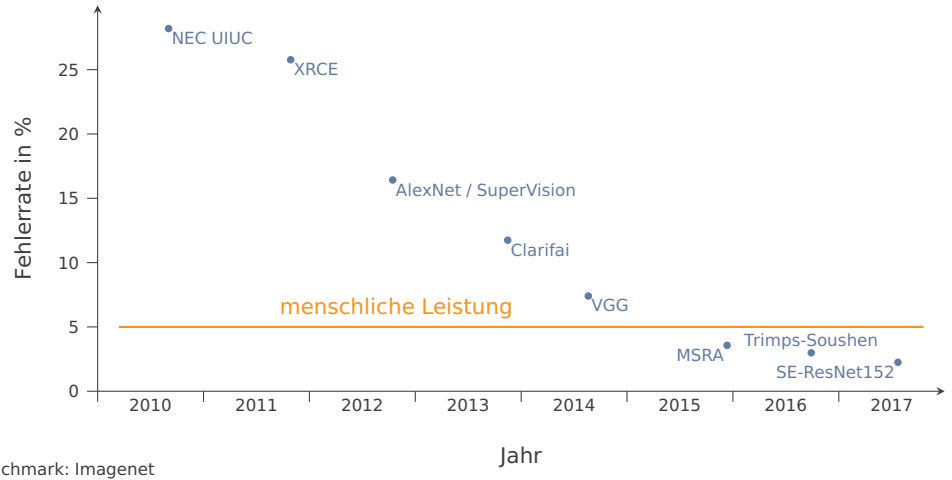

Datenquelle: John C. McCallum, eigene Darstellung bis 2018, https://www.guru3d.com

# Taktgeschwindigkeit von Intel-Prozessoren




Datenquelle: Wikipedia 2022

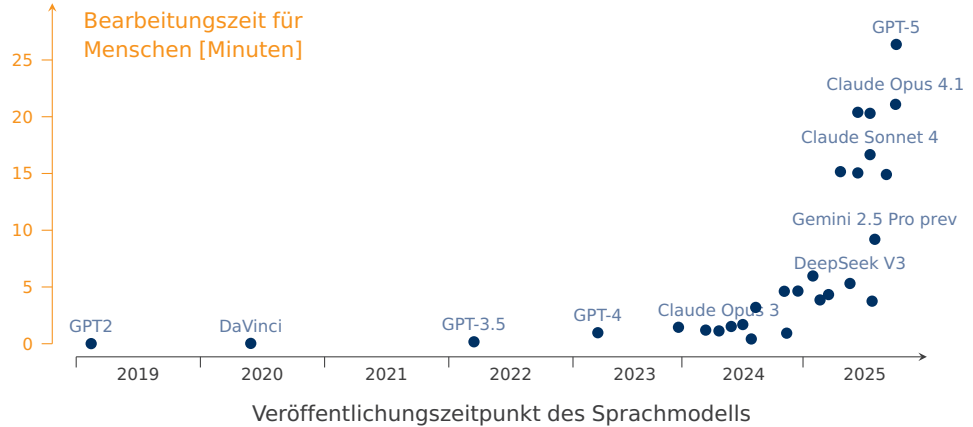
# Fortschritt in der Bilderkennung




Benchmark: Imagenet
Datenquelle: Electronic Frontier Foundation 2018

# Fortschritt in der Programmgenerierung

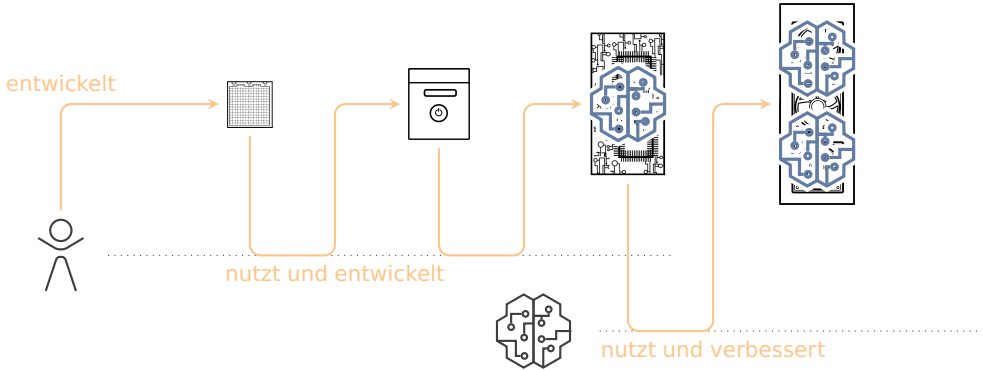
Aufgaben, die verschiedene Sprachmodelle mit 80 % Wahrscheinlichkeit lösen können




Kwa et al. Measuring AI Ability to Complete Long Tasks, 2025, `https://arxiv.org/abs/2503.14499`, eigene Darstellung

# Technischer Fortschritt mit Prognose

Existierende Rechner sind Werkzeuge zum Entwurf neuer Rechner.



# Syllabus

## Wintersemester 2026/27

| | | |
|---|---|---|
| 07.10.26 | 1. Einführung | |
| 14.10.26 | 2. Kombinatorische Logik I | |
| 21.10.26 | 3. Kombinatorische Logik II | |
| 28.10.26 | 4. Sequenzielle Logik | → Judith Senn |
| 04.11.26 | 5. Arithmetik I | |
| 11.11.26 | 6. Arithmetik II | |
| 18.11.26 | 7. Befehlssatzarchitektur (ARM) I | |
| 25.11.26 | 8. Befehlssatzarchitektur (ARM) II | |
| 02.12.26 | 9. Prozessorarchitekturen | |
| 09.12.26 | 10. Speicher | → Judith Senn |
| 16.12.26 | 11. Ein-/Ausgabe | |
| 13.01.27 | 12. Leistung | |
| 20.01.27 | 13. Ausblick: Quantencomputer | |
| 27.01.27 | 14. Fragestunde | → Übungsleiter:innen |
| **10.02.27** | **Klausur (1. Termin)** | |